

Microservices: Yesterday, Today and Tomorrow

Alessandro Gilardi 866035

Mattia Volpato 866316

Qazim Toska 847361

Architettura del Software - in-depth study

Indice

1	Introduzione	3
1.1	Definizioni	3
1.2	Problematiche e soluzioni	3
2	Yesterday	4
2.1	L'architettura del software prima dei servizi	4
2.2	La prima generazione di servizi: SOC e SOA	4
2.3	La seconda generazione di servizi: i Microservizi	5
3	Today	5
3.1	Differenze tra microservizi e SOA	5
3.2	Principi fondamentali dei microservizi	6
3.3	Caratteristiche dei microservizi	6
3.3.1	Punti di forza	6
3.3.2	Punti di debolezza	6
3.4	Impatto sugli attributi di qualità del software	7
4	Tomorrow	8
4.1	Affidabilità	8
4.1.1	Interfacce	8
4.1.2	Specifiche Comportamentali e Coreografie	8
4.2	Sicurezza	8
4.2.1	Ampia Superficie d'Attacco	8
4.2.2	Complessità della Rete	8
4.2.3	Fiducia	9
4.2.4	Eterogeneità	9
5	Conclusione	9

1 Introduzione

Nella sezione introduttiva l'articolo discute le principali problematiche delle architetture monolitiche, proponendo i *microservizi* come soluzione e riportando tutti i vantaggi che si possono ottenere con l'utilizzo di questi.

1.1 Definizioni

- *Monolite*: applicazione software i cui moduli non possono essere eseguiti indipendentemente.
- *Microservizio*: processo coesivo e indipendente che comunica attraverso messaggi.
- *Architettura a microservizi*: applicazione distribuita in cui tutti i moduli sono microservizi.

1.2 Problematiche e soluzioni

1. Difficoltà di manutenzione ed evoluzione:

- *Monoliti*: i monoliti di grandi dimensioni sono difficili da mantenere ed evolvere a causa della complessità della vasta code base. Inoltre, l'aggiunta o l'aggiornamento di librerie può causare incongruenze, rendendo il sistema instabile o addirittura non funzionante.
- *Microservizi*: al contrario, i microservizi sono composti da unità piccole e indipendenti di codice, caratteristiche che facilitano l'evoluzione delle funzionalità, l'aggiornamento e l'integrazione di librerie e la risoluzione di bug.

2. Scarsa flessibilità nell'avvio:

- *Monoliti*: qualsiasi modifica a un modulo richiede il riavvio dell'intera applicazione, con conseguenti tempi di inattività significativi.
- *Microservizi*: ogni microservizio può essere riavviato o aggiornato separatamente, riducendo i tempi di fermo e migliorando l'efficienza dello sviluppo e della manutenzione.

3. Difficoltà nel deployment e nella scalabilità:

- *Monoliti*: a causa dei requisiti contrastanti tra i moduli, nei monoliti non è sempre possibile ottimizzare il processo di deployment su infrastrutture dedicate. Inoltre, per garantire la scalabilità è necessario creare multiple istanze dell'intera applicazione, quando potenzialmente sarebbe sufficiente multistanziare solo determinati moduli.
- *Microservizi*: i microservizi permettono un deployment più flessibile e mirato, poiché ogni modulo può essere gestito da infrastrutture dedicate alle sue specifiche necessità di risorse. Inoltre, per garantire la scalabilità è possibile creare multiple istanze solo dei microservizi che gestiscono la maggior parte del carico di lavoro.

4. Lock-in tecnologico:

- *Monoliti*: i monoliti vincolano gli sviluppatori a utilizzare le stesse tecnologie e linguaggi del sistema originale.
- *Microservizi*: i microservizi permettono una maggiore libertà tecnologica, consentendo agli sviluppatori di scegliere gli strumenti e i linguaggi più adatti per ciascun servizio.

2 Yesterday

In questa sezione viene fornita una breve panoramica sull'evoluzione della ricerca nel campo dell'architettura del software.

2.1 L'architettura del software prima dei servizi

- Anni '60 e '70: comparsa dei primi problemi legati allo sviluppo di software su larga scala, che hanno stimolato un crescente interesse per il design e l'architettura del software.
- Anni '80: diffusione del concetto di architettura software, con la creazione di una prima definizione nel 1992.
- Anni '90: il paradigma **Objected Oriented Programming** (*OOP*) introduce il concetto di *pattern* come soluzione a problemi ricorrenti per la progettazione di software, tra cui il modello **Model-View-Controller** (*MVC*).

2.2 La prima generazione di servizi: SOC e SOA

Service Oriented Computing (*SOC*) è un paradigma per il calcolo distribuito, introdotto per gestire la complessità dei sistemi distribuiti e per integrare diverse applicazioni software. Questo paradigma di calcolo viene adottato dalla **Service Oriented Architecture** (*SOA*), stile architetturale in cui i programmi sono *servizi* che offrono funzionalità ad altri componenti attraverso l'utilizzo di messaggi, permettendo la separazione delle loro interfacce dall'effettiva implementazione.

I principali vantaggi ottenuti con l'utilizzo di *SOC* e *SOA* sono:

- **Dinamismo e Scalabilità**: la ripartizione del carico sul sistema viene gestita lanciando nuove istanze dello stesso servizio.
- **Modularità e riusabilità**: servizi complessi possono essere costruiti combinando servizi più semplici, e gli stessi servizi possono essere riutilizzati in contesti diversi.
- **Sviluppo del software distribuito**: l'indipendenza dei servizi e la definizione di interfacce stabili consente l'utilizzo di più squadre per lo sviluppo parallelo delle diverse componenti di un sistema distribuito.
- **Integrazione di sistemi eterogenei e legacy**: l'adozione di protocolli standard per la comunicazione tra i servizi permette l'integrazione con sistemi legacy.

2.3 La seconda generazione di servizi: i Microservizi

L'architettura a microservizi può essere vista come una semplificazione dell'architettura *SOA*, che cerca di concentrarsi su servizi indipendenti che implementano singole funzionalità direttamente legate a capacità di business, permettendo di ridurre la complessità.

A tal fine, vengono abbandonati i complessi e pesanti protocolli *WS-** a favore dello stile architetturale **REST** che, grazie alla sua semplicità, leggerezza e l'utilizzo di HTTP come protocollo di comunicazione, facilita l'integrazione, la scalabilità e la gestione decentralizzata. **REST** riduce infatti la necessità di middleware complessi, favorendo un'interazione diretta tra i servizi tramite risorse identificabili da URL.

3 Today

L'architettura a microservizi rappresenta un paradigma emergente nella programmazione, basato sulla suddivisione di un'applicazione in piccoli servizi indipendenti che comunicano tramite meccanismi leggeri. Derivata dai principi di *SOA*, questa architettura permette di passare da flussi di lavoro inter-sistemici a unità indipendenti focalizzate su singole capacità aziendali.

Introdotta formalmente nel 2011, l'idea dei microservizi ha radici in approcci precedenti¹ e si distingue per la sua enfasi sulla scalabilità, la manutenibilità e la decomposizione funzionale dei sistemi complessi. Ogni servizio è progettato per essere indipendente nello sviluppo e nella distribuzione, promuovendo i due principi di:

- Basso accoppiamento.
- Elevata coesione.

3.1 Differenze tra microservizi e SOA

Nonostante si possano individuare diversi elementi in comune, le architetture a microservizi si distinguono da *SOA* per alcune caratteristiche uniche:

- **Dimensione dei servizi ridotta:** un'architettura a microservizi pone un maggiore focus sulla granularità dei suoi componenti, cercando di associare ogni servizio a una singola capacità aziendale, al fine di favorire estendibilità ed evolutività;
- **Delimitazione dei contesti dei servizi:** funzionalità correlate sono raggruppate in contesti autonomi implementati come servizi indipendenti, favorendo un'elevata coesione;
- **Indipendenza dei servizi:** la comunicazione tramite interfacce pubblicate garantisce indipendenza operativa dei singoli servizi, promuovendo un basso accoppiamento.

¹Per esempio, già prima del 2011 veniva utilizzata una sua versione chiamata *fine-grained SOA* in Netflix

3.2 Principi fondamentali dei microservizi

L'implementazione efficace dei microservizi si basa su alcuni principi chiave:

1. **Team cross-funzionali:** ogni team è responsabile dell'intero ciclo di vita del servizio, secondo il principio *"you build it, you run it"* introdotto da Amazon;
2. **Automazione totale:** al fine di permettere aggiornamenti frequenti e rapidi risulta essenziale l'adozione di strumenti di *continuous integration* e *continuous delivery*, che permettono di mantenere l'applicazione sempre funzionante;
3. **Coreografia anziché orchestrazione:** la collaborazione tra servizi avviene tramite meccanismi ad eventi e di *publish/subscribe*, eliminando la necessità di un controllo centralizzato.

3.3 Caratteristiche dei microservizi

3.3.1 Punti di forza

Tra i principali benefici che si possono ottenere dall'adozione di un'architettura a microservizi si annoverano:

- **Flessibilità:** i microservizi permettono una maggiore dinamicità nell'adattamento rapido ai cambiamenti del mercato, come conseguenza della maggiore capacità di evoluzione derivante dall'indipendenza delle singole componenti.
- **Modularità:** la suddivisione in diverse componenti permette uno sviluppo indipendente e parallelo di questi, particolarmente adatto nella gestione di sistemi complessi.
- **Evoluzione continua:** l'indipendenza dei componenti abilita la possibilità di aggiornare e migliorare continuamente i servizi senza rischiare compromettere il sistema nel suo complesso.
- **Eliminazione dei *single point of failure*:** l'utilizzo di servizi indipendenti elimina la presenza di punti centrali di fallimento, permettendo al sistema di continuare a funzionare (seppur con funzionalità ridotte) anche in presenza di fallimenti di componenti.

3.3.2 Punti di debolezza

I principali contro nell'adozione dei microservizi riguardano la complessità derivante dal dover gestire un sistema distribuito al posto un unico sistema centralizzato:

- **Complessità di integrazione:** L'uso di un sistema distribuito introduce complessità nell'integrazione delle componenti, rendendo il sistema vulnerabile ai problemi legati alla comunicazione tra i servizi;
- **Inconsistenza e duplicazione nelle basi di dati:** l'utilizzo di più basi di dati nella stessa applicazione (idealmente una per microservizio) introduce duplicazione per degli stessi dati; inoltre, le transazioni distribuite non risultano applicabili nella maggior parte degli scenari, costringendo gli sviluppatori a dover adottare un approccio *eventually consistence*.

Inoltre, l'assenza di una definizione standardizzata per il concetto di microservizi ha portato ad adottare per questo termine una varietà di interpretazioni e pratiche non sempre coerenti tra loro, come evidenziato da esperti del settore quali Fowler, Newman e Krause.

3.4 Impatto sugli attributi di qualità del software

L'adozione di un'architettura a microservizi influenza diversi aspetti qualitativi del software:

- **Disponibilità:**

- La disponibilità dell'applicazione dipende da quella dei singoli microservizi: è necessaria un'attenta progettazione al fine di ottimizzare dimensione e complessità dei servizi, al fine di minimizzare i fallimenti.
- Con una corretta progettazione e implementazione è possibile ottenere una maggiore resilienza complessiva.

- **Affidabilità:**

- Maggiore vulnerabilità a fallimenti parziali dovuta alla natura distribuita del sistema.
- Problemi di comunicazione tra servizi dovuti anche dall'ampio utilizzo della rete, su cui non è possibile avere controllo.

- **Manutenibilità:**

- Ampi benefici ottenuti dall'indipendenza dei servizi.
- Riduzione dei costi di modifica e facilitazione dell'aggiunta di nuove funzionalità.

- **Performance:**

- La comunicazione su rete comporta un overhead importante rispetto alle chiamate a procedure in memoria.
- L'impatto complessivo varia in base al grado di accoppiamento tra servizi: idealmente, componenti completamente indipendenti non subiscono alcun impatto dalle possibili problematiche legate alla rete.

- **Sicurezza:**

- L'utilizzo della rete pubblica rende spesso necessario l'utilizzo di crittografia (che comporta un ulteriore overhead) per trattare informazioni sensibili con formati come JSON e XML.
- Necessità di protezioni avanzate per l'autenticazione e la gestione dei dati, soprattutto con l'integrazione di terze parti.

- **Testabilità:**

- Diminuita la complessità nei test funzionali, con la possibilità di testare ogni componente in isolamento.

- Incrementata la complessità nei test di integrazione, dove le anomalie emergono dalle interazioni tra più servizi, di cui spesso non è semplice replicare l'esecuzione.

4 Tomorrow

I microservizi presentano sfide significative, come la gestione delle modifiche nelle interfacce dei servizi, la prevenzione di attacchi alla comunicazione di rete e la difficoltà intrinseca nella programmazione di sistemi distribuiti (rispetto ai monoliti), che vengono esplorate a seguire.

4.1 Affidabilità

4.1.1 Interfacce

La possibilità di utilizzare tecnologie diverse per i microservizi comporta una difficoltà nella definizione e verifica delle interfacce. Lo standard attuale è la documentazione informale, che rende però lo sviluppo di programmi client suscettibile ad errori. Strumenti come Jolie, Apache Thrift e Protocol Buffers cercano di standardizzare la definizione delle interfacce, ma non esistono ancora soluzioni per integrare completamente stili architetturali come REST o linguaggi dinamici.

4.1.2 Specifiche Comportamentali e Coreografie

Utilizzare le API come interfacce formali non garantisce compatibilità tra i servizi se questi non rispettano l'ordine dei messaggi durante le loro interazioni: alcuni esempi possono essere situazioni di deadlock o di invio di messaggi su stream chiusi.

Alcune soluzioni che stanno venendo studiate per questo problema sono:

- **behavioural types**: sono tipi che possono verificare la compatibilità tra servizi, come ad esempio i *session type* (già utilizzati in scenari reali). Tuttavia, l'utilizzo pratico di questi tipi è limitato dalle restrizioni che portano nello sviluppo dei servizi.
- **coerografie**: descrivono il comportamento globale dei sistemi e possono essere usate per generare implementazioni corrette per costruzione; tuttavia, presentano ancora problemi irrisolti nei modelli di deployment, dovuti ad esempio a comportamenti non deterministici.

4.2 Sicurezza

4.2.1 Ampia Superficie d'Attacco

Nei microservizi, le comunicazioni avvengono tramite API esposte alla rete, aumentando la superficie d'attacco rispetto ai monoliti.

4.2.2 Complessità della Rete

La decomposizione delle applicazioni in numerosi microservizi porta a una maggiore complessità nel monitoraggio e l'analisi della sicurezza, aumentando il rischio di attacchi.

4.2.3 Fiducia

I microservizi si basano spesso su un'ipotesi di fiducia reciproca. Questo approccio è problematico in contesti aperti, dove la compromissione di un singolo servizio può arrivare a compromettere dati sensibili di un intero sistema, come dimostrato in un caso in Netflix, in cui la compromissione di un microservizio ha causato la diffusione di dati sensibili degli utenti.

4.2.4 Eterogeneità

I sistemi basati su microservizi amplificano l'eterogeneità, includendo entità autonome, domini di sicurezza separati e interazioni attraverso API. L'assenza di un'infrastruttura comune rende difficile l'applicazione di regole di sicurezza globali.

5 Conclusione

I microservizi offrono opportunità promettenti ma richiedono una ricerca approfondita per affrontare le sfide legate a dipendenza, sicurezza e fiducia. Modelli formali e metodologie innovative come le coreografie possono rappresentare una base solida per il futuro, se adeguatamente adattati a scenari pratici. La complessità e l'apertura del paradigma richiedono uno sforzo congiunto per sviluppare sistemi sicuri e affidabili.